CHAPTER 2

# THE SOFTWARE PROCESS

# Overview

- Client, Developer, and User
- Requirements Phase
- Specification Phase
- Design Phase
- Implementation Phase
- Integration Phase
- Maintenance Phase
- Retirement
- Problems with Software Production: Essence and Accidents

# The Software Process

- The software process is the way we produce software. It incorporates the
  - life-cycle model
  - CASE tools (Computer-Aided Software Engineering)
  - The individuals building the software

- Building an app is a lot like **building a house.**
- To build a house, you can’t just throw bricks in a pile; you need a process. Here is how those three parts work:
-

**1. The Life-Cycle Model (The Blueprint)**

- This is the **step-by-step plan.**
- You don't paint the walls before you lay the foundation.
- The model decides the order: first you dig the hole, then you frame the house, then you do the roof.
- **The Goal:** It ensures you don't build things in the wrong order and have to start over.

- **2. CASE Tools (The Power Equipment)**
- These are the **modern machines** that make the job faster and safer.
- Instead of using a hand saw, you use a circular saw. Instead of a shovel, you use an excavator.
- In software, these are programs that automatically check for "cracks" (bugs) in the code or help designers draw the 3D layout.
- **The Goal:** To work smarter and faster than doing everything by hand.

- **3. The Individuals (The Crew)**
- These are the **skilled workers** on-site.
- You have the architect (Designer), the plumber (Backend Developer), and the electrician (Database Expert).
- Even with the best blueprint and the best tools, you still need someone who knows how to swing the hammer.
- **The Goal:** To have experts who know how to use the tools and follow the plan.

- **The Bottom Line:**
- If you have a **Blueprint** but no **Crew**, the house is never built. If you have a **Crew** but no **Blueprint**, you'll end up with a door that opens into a wall. You need all three for a solid house.

# Terminology

- Systems analysis
  - Requirements + specifications phases
- Operations mode
  - Maintenance
- Design
  - Architectural design, detailed design
- Client, developer, user
- Internal software development
- Contract software development (outsourcing)

# Testing Phase?

- There is NO testing phase separately
- Testing is an activity performed throughout software production
- Verification
  - Performed at the end of each phase
- Validation
  - Performed before delivering the product to the client

- **1. Testing as a Constant Activity**
- Instead of waiting until the house is finished **to see if the lights work**, the electrician tests the wires **while** they are installing them.
- In software, developers test small pieces of code the moment they write them.

- 2. **Verification: "Checking the Blueprint"**
- "Are we building the thing right?" This happens at the end of every small step to ensure you followed the plan correctly.
- **Real-Life Example:** After the foundation of the house is poured, an inspector comes by. They check if the concrete is the right thickness and if it matches the architectural blueprint.
- **The Goal:** You don't move to the next step (building walls) until you are 100% sure the current step (the floor) is perfect.

- **3. Validation: "Checking with the Owner"**
- This happens at the **very end**, right before you hand over the keys. You are checking if the house actually meets the owner's needs.
- **Real-Life Example:** The homeowner walks through the kitchen and says, "I know the blueprint shows the sink here, but it's too high for me to reach comfortably."
- **The Goal:** To make sure you "built the right thing" for the

# Documentation Phase?

- There is NO documentation phase separately
- Every phase must be fully documented before starting the next phase
  - Postponed documentation may never be completed
  - The responsible individual may leave
  - The product is constantly changing—we need the documentation to do this
  - The design (for example) will be modified during development, but the original designers may not be available to document it

- **Restaurant Chef.**
- In a professional kitchen, documentation isn't a separate task done at the end of the day; it happens *while* the food is being prepped.

- **1. No Separate Phase (Write while you cook)**
- You don't cook a new 5-course meal and then try to remember every pinch of salt you used two hours later.
- **The Reality:** You write the recipe (the documentation) as you experiment with the ingredients. If you wait until the end, you'll forget the exact temperature or the "secret" spice you added.

- **2. The "Leaver" Problem (The Chef Quits)**
- Imagine the Head Chef creates a world-famous sauce but says, "I'll write down the recipe tomorrow".
- **The Disaster:** That night, the Chef gets a better job and leaves.
- **The Result:** Now, the restaurant has a "product" (the sauce) that no one knows how to make anymore. Without the documentation, the business is in trouble.

- **3. The Product Changes (The Recipe Update)**
- Halfway through the month, the Chef decides the sauce is too salty and changes the salt to soy sauce.
- **The Need:** If the recipe book isn't updated **immediately**, the assistant chefs will keep making the old, salty version.
- **The Goal:** Documentation must change every time the "product" changes so the team stays consistent.

**Summary in a Nutshell:**

- **Documentation is like a Recipe Book:** If you don't write the steps down *while* you're cooking, the next person in the kitchen won't be able to finish the meal or fix it if it tastes bad.

# Requirements Phase

Should we build this software?

**1 Assumption**

- Software must be worth the money
- Benefit or profit should be more than cost

**2 Client Needs**

- Focus on what is **really needed**, *not* what the client wants.
- Avoid extra features that are not important.

**3 Constraints (Limits)**

- **Deadline** – When must it be finished?
- **Reliability** – Should work without failure
- **Code Size** – Must fit in device memory (for small systems)
- **Cost** – Must stay within budget

**4 Step-by-Step Improvement**

- Idea is improved little by little
- Each feature is checked: **Can we build it?  Can we afford it?**

**🎯 Goal:**

Make sure the project is **possible, useful, and affordable** before starting development.

# Requirements Phase Testing

- **Problem:**
  Clients often **don't know exactly what they need** until they see the system.

- **Solution:**
  **Rapid Prototype** = A quickly built demo version of the software.

**Purpose:**

✓ Shows main features
✓ Helps client understand the system
✓ Makes requirements clearer

- **Does NOT show:**

✗ Internal coding

✗ Database structure

✗ Security

✗ Performance

## Example:

A college website made to show layout and features before final development.

- Before building the real college website, developers create a **sample/demo website.**

This demo shows:

- How the homepage looks
- Where menus are placed
- How course pages appear
- How contact forms work

So the college staff can see it and say:

- "Move this menu"
- "Add student login"
- "Change colours"
- After feedback, the **final website** is built correctly.

☞ It is just for showing **layout and features**, not the full working system.

- Before building the real college website, developers create a **sample/demo website.**

This demo shows:

- How the homepage looks
- Where menus are placed
- How course pages appear
- How contact forms work

So the college staff can see it and say:

- "Move this menu"
- "Add student login"
- "Change colours"
- After feedback, the **final website** is built correctly.

☞ It is just for showing **layout and features**, not the full working system.

# Requirements Phase Documentation

**Requirements are recorded in one of two ways:**

1 **Rapid Prototype + Discussion Notes**

- A demo version of the system is shown
- Client feedback is discussed
- All decisions and changes are written down

**OR**

2 **Requirements Document (SRS)**

A written document describing:

✓ System features (View food menu, View restaurants, User registration & login)

✓ User needs (Users need order tracking, Users need online payment, Users need to place orders)

✓ Constraints (Reliability, Internet limit, Device limit, Cost, Deadline)

**Checking Process**

✓ **Client & Users** – Confirm it matches their needs

✓ **Development Team** – Check if it can be built

✓ **SQA Team** – Ensure requirements are clear, complete, and correct

🎯 **Goal:** Everyone agrees on what to build before development starts.

# Software Quality Assurance (SQA)

•Makes sure the final software is **what the client asked for**
•Ensures the software is **developed correctly**

Checks that:
✓ Requirements are clear
✓ Standards are followed
✓ Testing is done properly

•SQA team is involved **from the beginning** of the project, not only at the end

**Moving Target Problem**

- Happens when the **client keeps changing requirements** during development
- Client may change ideas after seeing progress
- This causes:

✗ Delay in project

✗ Increased cost

✗ Confusion for developers

# Specification Phase

A written document that clearly defines **what the software must do.**

**Contains**

✓ System features and functions

✓ Inputs to the system

✓ Outputs from the system

✓ Acts as a **legal agreement** between client and developer

**Avoid Vague Words**

Do NOT use terms like:

✗ Suitable

✗ Enough

✗ Optimal

✗ 98% complete

**Goal:**

Provide clear, exact, and measurable requirements to avoid confusion.

**(Food Delivery App)**

✗ *Wrong:* "App should load fast"

✓ *Correct:* "App must load within **3 seconds** on 4G network"

# Specification Phase

**Specifications Must Be Clear**

The specifications document should NOT be:

❌ **Ambiguous** – unclear meaning

❌ **Incomplete** – missing information

❌ **Contradictory** – two parts saying different things

👉 Everything must be clear, complete, and consistent.

✅ **After Specifications Are Approved**

Once the client and team **agree and sign** the specifications:

➡ The **Software Product Management Plan (SPMP)** is prepared.

**What is SPMP?**

SPMP is the **project plan**. It tells:

✓ What tasks need to be done

✓ Who will do each task

✓ When each task must be completed (deadlines)

# SPMP (Software Project Management Plan)

- A document that shows **how the software project will be carried out.**

**Includes**

✓ **Deliverables** – What the client will get
✓ **Milestones** – When the client will get them

**Also Tells**

- How the software will be developed
- Who is in the team and their roles
- What tasks need to be done
- Tools and methods used
- Budget and resources needed

**Most Important**

 Time schedule

💰 Cost estimation

# Specification Phase Testing

**Traceability**

- Every line in the specification must come from a **client requirement**
- We should be able to **track each feature back to what the client asked**

**Review**

- Done to check if the **specification is correct and complete**
- **Specification team + Client** take part
- **SQA (Software Quality Assurance)** member leads the review

**SPMP Check**

- The **SQA team also reviews the SPMP**
- Ensures proper planning, quality, time, and cost control

🎯 **In short:**
Make sure the specs match client needs and are checked properly.

# Specification Phase Documentation

☞ **Tells WHAT the software must do**

It is written based on client requirements.

**Includes:**

•Features of the system
•Functions the software must perform
•User requirements
•Constraints (rules, limits)
•Performance needs

**Example:**

For a Library System:
•Students can issue books
•Max 3 books per student
•Fine ₹5 per day late

• **It does NOT say how to build it.**

**SPMP (Software Project Management Plan)**

☞ **Tells HOW the project will be managed and developed**

Prepared **after specs are finalized.**

**Includes:**

•Tasks to be done
•Team members and responsibilities
•Schedule and deadlines
•Budget and cost
•Resources and tools used
•Milestones and deliverables

**Example:**

•Module 1 (Login) – Completed by Feb 10
•Testing team – 2 members
•Total project cost ₹2 lakhs

| Specification Document | SPMP |
| --- | --- |
| WHAT system should do | HOW project will be done |
| Technical requirements | Project planning |
| Written for developers | Written for managers/team |
| No schedule or cost | Has time, cost, resources |

# Design Phase

- **Specification = WHAT**
- Describes **what the system should do**
- Based on customer needs
- No technical details of building
- **Example:**
  "Online system should allow users to book tickets."

**Design = HOW**

- Describes **how the system will be built**
- Technical plan for developers

**Why keep design decisions?**

- Helps when the team gets stuck (dead-end)
- Useful for the **maintenance team** later
- Future developers understand why something was done

**Design should be open-ended**

- Easy to modify
- Easy to add new features lat

# Design Phase Testing

**Traceability**

•Every design part must match something in the **specification**

•Ensures the design is based on **real requirements**

•Nothing extra, nothing missing

**Example:**

Spec says: *"System must allow student login"*

Design must include: *Login module*

**Review**

•Design is checked for correctness

•Done by:

- 👩‍💻 Design Team
- ✅ SQA (Software Quality Assurance) group

•❌ Client is NOT involved in this review

**Purpose:**

To make sure the design is correct, clear, and follows the specification

# Design Phase Documentation

**1 Architectural Design**

☞ Break the whole system into major parts (modules)

Example:

- Login module
- Payment module
- Report module

**2 Detailed Design**

☞ Design each module in detail:

- Data structures
- Algorithms
- Logic

Example:

- How login checks username & password
- How payment is processed

# Implementation Phase

**Implement the detailed design in code**

- Developers convert the **design into a working program**
- Each module designed earlier is now **coded**
- Follow:
  - Design documents
  - Coding standards
- Output of this phase = **Source code**

**Example:**

Design says: *Login module with username & password check*

Implementation = Writing the actual **login program code**

**In short:**

Design on paper ➜ Turned into real software through coding

# Implementation Phase Testing

**Code Review**

- Programmer explains the code to the team
- Team checks:
  - Logic
  - Errors
  - Standards followed
- **SQA member** is part of the review

**Testing**

📝 **Informal Testing**

- Done by programmer
- Also called **desk checking**
- Finds small mistakes early

✅ **Formal Testing**

- Done by **SQA team**
- Uses planned **test cases**
- Checks if software works as required

🎯 **In short:**

Review the code ➜ Test it ➜ Make sure it works correctly

# Implementation Phase Testing

- Code Review
  - The programmer explains the code to the review team, which includes a SQA member
- Test cases
  - Informal testing (desk checking)
  - Formal testing (SQA)

# Implementation Phase Documentation

- Source code
  - Must have suitable comments
- Test cases (with expected output)

# Integration Phase

**Code Review**

- Programmer explains the code to the review team
- Team includes **SQA member**
- Purpose: find mistakes, improve quality

**Testing**

- **Test cases** are prepared to check the program
- **Types of Testing:**
- 📝 **Informal Testing** – Desk checking by programmer
- 🖥️ **Formal Testing** – Done by SQA team

🎯 **In short:**

Check the code ➜ Test the program ➜ Ensure it works correctly

# Integration Phase Testing

**Product Testing**

- Done by **Integration Team + SQA**
- Checks if all parts of the software work together correctly

**Acceptance Testing**

- Done by the **Client**
- Software is tested on **real hardware** with **real data**
- Confirms the product meets client requirements

🎯 **In short:**
Team tests the system ➜ Client tests it ➜ Software is approved for use

# Integration Phase Documentation

**Commented Source Code**

- Program code with explanations (comments)
- Helps others understand the logic
- Makes future maintenance and updates easier

**Test Cases (Complete Product)**

- List of tests for the entire system
- Shows expected input and output
- Proves the software works correctly

🎯 **In short:**
Clear code + Proper testing records = Easy maintenance & reliable software

# COTS Software

**1. Buying Software: COTS**

- **Commercial Off-The-Shelf (COTS)** refers to software that is ready-made and available to the general public.
- **Old School: "Shrink-wrapped"**
  - Think of physical boxes you'd buy at a store (like Office or Windows on a disc). Opening the plastic wrap meant you accepted the license.
- **Modern Day: "Click-wrapped"**
  - Now, we download software instantly. You "accept" the terms by clicking a button before installation.

**2. Testing Software: The Roadmap**

Before software is sold to everyone, it goes through two critical "real-world" checkups:

| Phase | Who tests it? | Goal |
|---|---|---|
| **Alpha Testing** | A small, selected group of potential clients. | To find major bugs and see if the core idea works. |
| **Beta Testing** | A larger group of users (the "corrected" version). | To polish the software and ensure it's ready for the final release. |

# Maintenance Phase

- Maintenance is the longest and most expensive stage of the software lifecycle.
- **What is it?**

Any change or update made after the client accepts the software.

- **The Cost:**

This phase consumes the most money in the entire development process.

- **The Biggest Hurdle:**

A "lack of documentation" often makes it difficult and costly to update old code.

# Maintenance Phase Testing

- Maintenance is the **longest and most expensive** phase of the software life cycle. It begins the moment the client accepts the product.

**1. The Core Challenges**

- **Ongoing Changes:** Includes any modification made after the initial release.
- **High Cost:** This phase consumes the majority of a project's total budget.
- **Documentation Gap:** A lack of clear technical records often makes it difficult and costly for new teams to update the code

**2. Ensuring Quality: Regression Testing**

- When you change software, you must ensure you haven't accidentally broken what was already working.
- **Verification:** Confirming new changes are implemented correctly.
- **Regression Testing:** Re-running previous test cases

# Example of Regression testing

example using a **Phone Update:**

- **The Example: Adding "Dark Mode"**
- Imagine your phone gets an update to add a new **Dark Mode** feature.
- **The Change:** The developers add code so your screen can turn black and grey.
- **The New Test:** You turn on Dark Mode to see if the colors changed correctly.
- **The Regression Test:** You open your **Camera** or your **Alarm Clock** to make sure they still work.
  - **The Logic:** Even though the update was about *colors*, you need to make sure the change didn't accidentally break your ability to take a photo or wake up on time.

# Maintenance Phase Documentation

- **Change Records**: A list of all edits and the **reasons** behind them.
- **Regression Test Cases**: A saved library of tests to use after every update.

# Retirement:The Final Act

While software is built to be maintained, it eventually reaches a point where it can no longer be updated.

**1. Retirement vs. Rewriting**

- **Good software is maintained:** As long as it's useful, it's kept alive through updates.
- **True retirement is rare:** Most software doesn't just "die"; it is usually replaced or evolved.

**2. When is it time to start over?**

- Sometimes software becomes **unmaintainable**, and it is cheaper to rewrite it from scratch than to keep fixing it. This happens when:
- **Drastic Design Changes:** The old structure can't handle new requirements.
- **Platform Shifts:** The software needs to run on completely new hardware or operating systems.
- **The Documentation Trap:** If documentation is missing or wrong, no one knows how to fix the old code safely.

# Example

- **Think of a Classic Car:** You can maintain it (oil changes, new tires) for decades. But if the engine explodes and the frame is rusted through (unmaintainable), it's often cheaper to buy a brand-new car than to try to rebuild the old one.

# Process-Specific Difficulties

- Does the product meet the user's real needs?
- Is the specification document free of ambiguities, contradictions, and omissions?

# Inherent Problems of Software Production

Building software is naturally difficult. Fred Brooks famously stated there is **"No Silver Bullet"**— no single tool that makes software development easy.

**Why Software is Hard (Inherent Limits) or Aristotelian Categories Essence:** Difficulties that are part of the very nature of software.

**Essence (Essential Difficulties)**

- These come from the **nature of software itself**. You cannot remove them — they are unavoidable.
- They exist because software deals with:
- **Complexity**
  - Many components, interactions, conditions
  - Real-world problems are messy
- **Conformity**
  - Software must match external rules
  - Business laws, hardware limits, user needs
- **Changeability**
  - Requirements keep changing
  - Software must evolve
- **Invisibility**
  - Software cannot be seen physically

- **Accidents (Accidental Difficulties)**
- These are caused by **tools, languages, methods, or environment** — not by the core problem.

Examples:

- Poor programming languages
- Bad tools or IDEs
- Low-level memory management
- Inefficient processes
- Hardware limitations
- Debugging difficulties due to tools

**These can be reduced with:**

- **Better languages (Python, Java)**
- **Modern IDEs**
- **Frameworks eg.** Instead of writing full authentication code, you use built-in:

```
from django.contrib.auth import authenticate.
```

- **Automation** Tools like:

GitHub Actions

Jenkins

# Complexity

**Why Software is Hard (Fred Brooks)**

- **No Silver Bullet:** There is no single invention that makes software easy to build.
- **Essential Complexity:** Software is naturally complex, invisible, and constantly changing.
- **The Problem:** Because we cannot understand the "whole" of a complex system, it is hard to manage and maintain.
- **Retirement:** True retirement is rare; software is usually rewritten only if it becomes "unmaintainable".

# Conformity

- Conformity refers to the requirement that software must "conform" to complex, pre-existing environments that it did not create.
- **Forced Complexity:** Unlike the laws of physics, which are consistent, software must adapt to inconsistent human systems, such as laws, business rules, and legacy hardware interfaces.
- **The "User" Problem:** Software must conform to what the user *actually* needs, which can be difficult to define or change frequently.
- **No Universal Standard:** Because there is no single "natural law" for how a business process or interface should work, the software becomes a "patchwork" of rules to fit into these different environments.

**Why it's a Problem**

- **Complexity:** This constant need to fit into "unnatural" systems increases the overall complexity of the software.
- **Maintenance Nightmare:** Because the systems the software must conform to are always changing (e.g., a new tax law or a new OS update), maintenance becomes a constant, expensive task.
- **Inherent Difficulty:** Brooks classifies this as an **"Essence"**—a difficulty inherent in the nature of

- A banking app must follow many outside rules.

### 1 Government Rules

- If the government changes tax laws or KYC rules → the app must update.
- The app didn’t create these laws, but must follow them.

### 2 Old Bank Systems

- Banks still use old computers and databases.
- The new app must connect with these old systems, even if they are outdated.

### 3 Different Users

- Young users want fast mobile banking
- Elderly users want simple screens
- Business users want advanced features
- The app must satisfy everyone.

### 4 No Single Standard

- Every bank and country has different rules.
- So software becomes a mix of many rules.

# Changeability

- In software engineering, **Changeability** refers to the constant pressure to modify software because it is perceived as "soft" and easier to alter than physical hardware.

**Why Changeability is a Problem**

- **Constant Pressure:** Because software can be changed, users constantly request new features and updates to keep it "useful".
- **Long Lifespan:** Useful software often lives for about **15 years**, whereas hardware is typically replaced after about **4 years**.
- **The Maintenance Trap:** This long lifetime means software spends most of its existence in the **Maintenance phase**, which is the most expensive part of the lifecycle.

**Managing the Changes**

- To handle constant changes without breaking the system, developers must use:
- **Regression Testing**: Re-running old tests to ensure new changes didn't break existing features.
- **Strict Documentation**: Keeping a record of every change made and the **reason** why it was done.